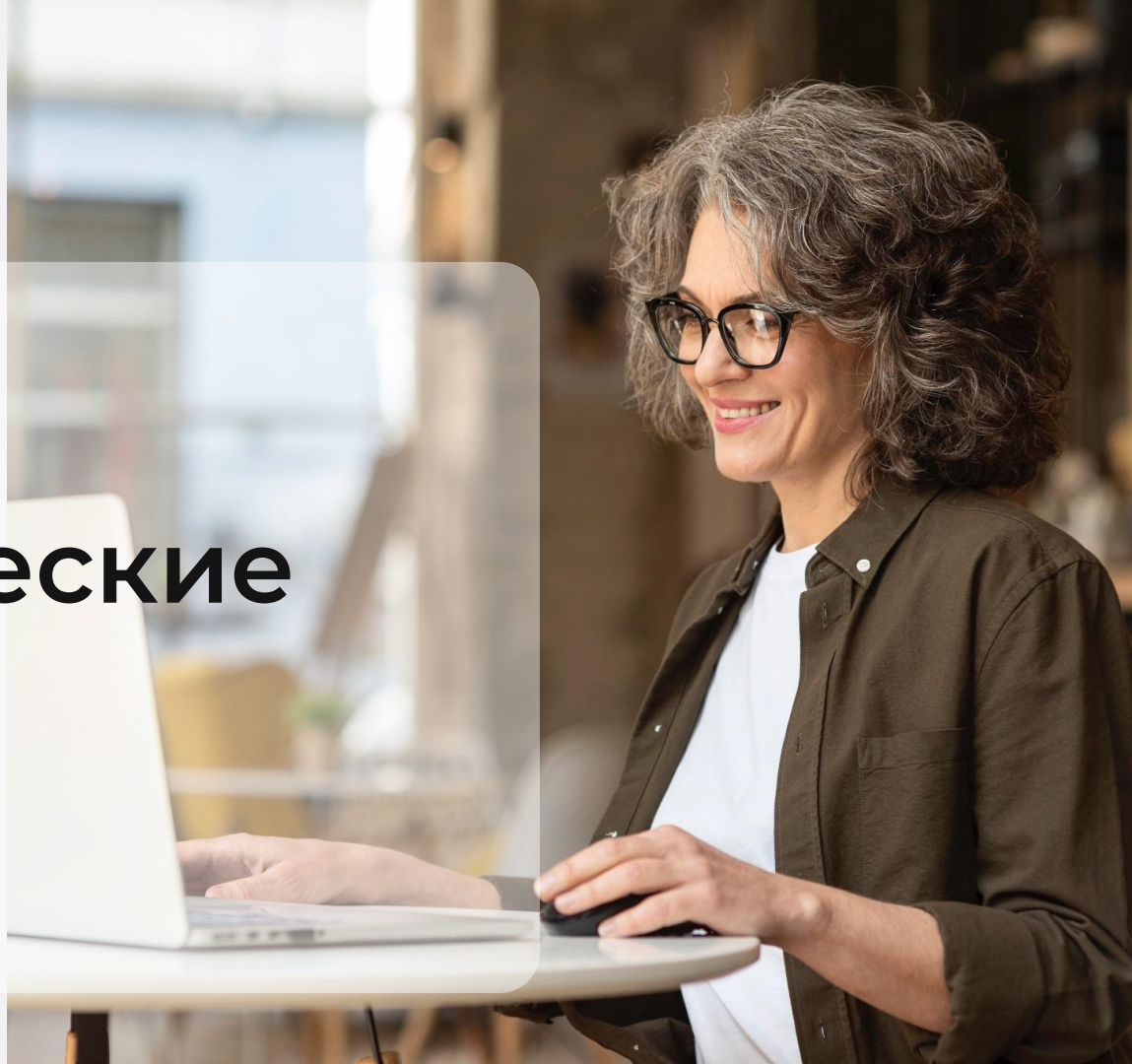


Python

Арифметические операторы, выражения



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

- Объекты и типы данных
- Примитивные типы данных
- Возвращаемое значение
- Передача аргументов в функцию
- Функция type
- Строковые операции
- Работа с кавычками в строках
- Экранирование символов
- Параметры sep и end

План занятия

- Числа
- Арифметические операции с числами
- ZeroDivisionError
- Приоритет математических операций и скобок
- Неизменяемые типы данных
- Операторы приращения
- Множественное присваивание
- Преобразование типов, ValueError



ОСНОВНОЙ БЛОК





Арифметические операции с числами

В Python числа представлены типами:



Целые числа (int)



Числа с плавающей точкой или вещественные (float)



ВОПРОСЫ





Арифметические операторы



Арифметические операции

Python поддерживает стандартные арифметические операции, такие как сложение, вычитание, умножение и деление.

Полезно знать



Операция

```
print("Сложение:", "3 + 5 =", 3 + 5)
print("Вычитание:", "7 - 2 =", 7 - 2.0)
print("Умножение:", "4 * 6 =", 4 * 6)
```

Деление

```
print("Деление:", "8 / 2 =", 8 / 2)
print("Целочисленное деление:", "7 // 2 =", 7 // 2)
print("Остаток от деления:", "7 % 3 =", 7 % 3)
```

Полезно знать



Переменная

Если какой-то результат в последующем нужно использовать повторно, то можно поместить его в переменную.

```
sum1 = 3 + 5
a = sum1 + 1
b = sum1 * 3
print(a, b)
```

Повторение

Если какое-то действие нужно совершить один раз, то хранить информацию в переменной бессмысленно.

```
print(3 + 5)
```



ВОПРОСЫ





ZeroDivisionError



ZeroDivisionError

Это исключение, которое возникает когда происходит попытка деления на ноль.

Когда возникает ZeroDivisionError?



Пример

- При делении целых чисел на ноль
- При делении вещественных чисел на ноль
- При использовании оператора остатка от деления на ноль

Код

```
a = 10
b = 0
result = a / b
```



ВОПРОСЫ





Приоритет математических операций и скобок

Экспресс-опрос

?

Какое арифметическое действие выполняется первым?

?

В скобках или без?



Скобки

В Python, как и в математике, операции выполняются в определенном порядке. Это называется приоритетом операций.

Общие правила приоритета операций:



Скобки (())

Операции внутри скобок всегда выполняются первыми, независимо от их приоритета.

Пример

```
result = (2 + 3) * 4 # Скобки  
заставляют сложить 2 и 3 перед  
умножением
```

```
print(result) # Результат: 20
```

Общие правила приоритета операций:



Возведение в степень (**)

После скобок, возведение в степень имеет наивысший приоритет.

Пример

```
result = 2 * 3 ** 2
# Это интерпретируется как 2 * (3 ** 2)
print(result)           # Результат: 18
```

Приоритет операций

Приоритет	Операция	Символы
1	Скобки	()
2	Возведение в степень	**
3	Умножение, Деление, Целочисленное деление, Остаток от деления	*, /, //, %
4	Сложение и Вычитание	+, -



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(5 + 3 * 2)
```

- a. 16
- b. 11
- c. 13
- d. 10



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(5 + 3 * 2)
```

- a. 16
- b. 11**
- c. 13
- d. 10



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(11 % 4)
```

- a. 1
- b. 2
- c. 3
- d. 4



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(11 % 4)
```

- a. 1
- b. 2
- c. 3**
- d. 4



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(2 * 3 ** 2)
```

- a. 12
- b. 36
- c. 18
- d. 9



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
print(2 * 3 ** 2)
```

- a. 12
- b. 36
- c. 18**
- d. 9



Выберите правильный вариант ответа

Что произойдёт при попытке выполнить следующую операцию?

```
print(7 // 0)
```

- a. 0
- b. 7
- c. Ошибка ZeroDivisionError
- d. 1



Выберите правильный вариант ответа

Что произойдёт при попытке выполнить следующую операцию?

```
print(7 // 0)
```

- a. 0
- b. 7
- c. Ошибка ZeroDivisionError**
- d. 1



ВОПРОСЫ





Неизменяемые типы данных



Неизменяемые типы данных

В Python существуют неизменяемые (immutable) и изменяемые (mutable) типы данных. Неизменяемые типы данных не могут быть изменены после создания объекта, а изменяемые могут.

Неизменяемые типы данных



Пояснение

То есть если у нас есть переменная `num` равная 5, то мы не можем изменить значение этой переменной, например с помощью арифметической операции:

Пример

```
num = 5
num + 2          # В переменной num
останется значение 5
print(num)
```

Неизменяемые типы данных



Пояснение

Для того чтобы изменить значение в num нужно присвоить переменной новое значение.

Пример

```
num = 5
num = num + 2      # В переменную num
будет присвоен результат вычисления 5 +
2
print(num)
```



ВОПРОСЫ





Операторы приращения



Операторы приращения

Это операторы, которые изменяют значение переменной на основе её текущего значения.

Операторы приращения



Прибавление (+=)

```
a = 5
a += 1 # Эквивалентно: a = a + 1
print(a) # Результат: 6
```

Умножение (*=)

```
a = 5
a *= 2 # Эквивалентно: a = a * 2
print(a) # Результат: 10
```



ВОПРОСЫ





**Множественное
присваивание**



Множественное присваивание

Это особенность Python, которая позволяет присваивать значения нескольким переменным одновременно в одной строке.

Присваивание нескольких значений



```
a, b, c = 1, 2, 3 # Инициализируем переменные a, b, c значениями 1, 2, 3 соответственно  
print(a, b, c)   # Вывод: 1 2 3
```

Присваивание одинакового значения нескольким переменным



```
a = b = c = 0      # Инициализируем каждую из переменных a, b, c значением 0  
print(a, b, c)     # Вывод: 0 0 0
```


Обмен значениями между переменными



```
a, b = 5, 10  
a, b = b, a # Меняем местами значения a и b  
print(a, b) # Вывод: 10 5
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какое значение будет у переменной num после выполнения следующего кода?

```
num = 5
```

```
num + 3
```

```
print(num)
```

- a. 8
- b. 5
- c. Ошибка
- d. 3

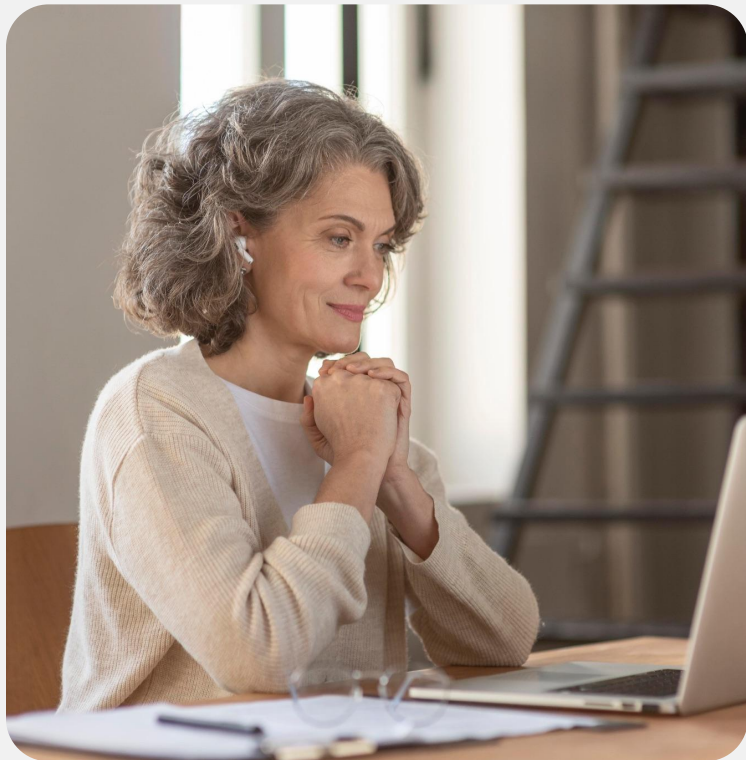


Выберите правильный вариант ответа

Какое значение будет у переменной num после выполнения следующего кода?

```
num = 5
num + 3
print(num)
```

- a. 8
- b. 5**
- c. Ошибка
- d. 3



Выберите правильный вариант ответа

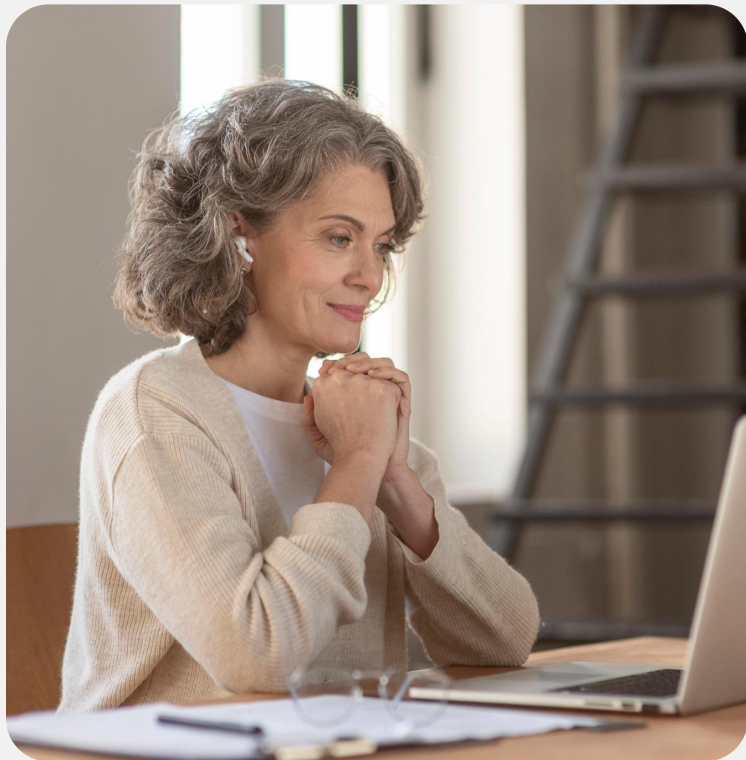
Какое значение будет у переменной `num` после выполнения следующего кода?

```
a, b = 1, 2
```

```
a, b = b, a
```

```
print(a, b)
```

- a. 1, 2
- b. 2, 1
- c. 3, 3
- d. Произойдет ошибка



Выберите правильный вариант ответа

Какое значение будет у переменной `num` после выполнения следующего кода?

```
a, b = 1, 2
```

```
a, b = b, a
```

```
print(a, b)
```

a. 1, 2

b. 2, 1

c. 3, 3

d. Произойдет ошибка



ВОПРОСЫ





Преобразование типов, ValueError

Экспресс-опрос

?

Какие типы данных бывают?



Преобразование типов

Это процесс преобразования значения одного типа данных в значение другого типа.

Преобразование типов



```
num1 = "10"  
num2 = "5.0"  
print(num1 + num2)  # Конкатенация строк  
print(num1 / num2)  # Попытка выполнить деление строк
```



Преобразование типов

При попытке выполнить деление `num1` на `num2`, Python вызывает ошибку `TypeError`, поскольку оператор `/` не поддерживается для строк.

Преобразование типов



```
num1 = "10"
num2 = "5.0"
result1 = int(num1) + float(num2) # Приведение строк к числовым типам и выполнение сложения
result2 = int(num1) / float(num2) # Приведение строк к числовым типам и выполнение
деления
print(result)                    # Вывод: 15.0 2.0
```



Явное преобразование

Происходит, когда разработчик вручную преобразует один тип данных в другой с помощью встроенных функций.

Целое число в строку



```
a = 5
b = str(a)      # Преобразуем целое число в строку
print(b)        # Вывод: '5'
print(type(b))  # Вывод: <class 'str'>
```




Неявное преобразование

Python может автоматически преобразовывать некоторые типы данных в других контекстах, например, при выполнении арифметических операций между числами разных типов.

Неявное преобразование



```
result = 5
print(type(result))    # <class 'int'>
result += 3.14         # Неявное преобразование int в float
print(type(result))    # <class 'float'>
```

Преобразование строки в число



```
a = "10"
b = int(a)      # Преобразуем строку в целое число
print(b)        # Вывод: 10
print(type(b))  # Вывод: <class 'int'>
```

Преобразование вещественного числа в целое



```
a = 10.7
b = int(a) # Преобразуем вещественное число в целое
print(b)  # Вывод: 10 (дробная часть отбрасывается)
```

Преобразование разных типов



```
a = "abc"
b = int(a) # Ошибка: ValueError
```

Не все типы могут быть преобразованы друг в друга

Преобразование вещественных чисел



```
a = 5.99
b = int(a)
print(b) # Вывод: 5
```

При преобразовании вещественных чисел в целые дробная часть отбрасывается

Преобразование строк



```
a = "False"
b = bool(a)
print(b)  # Вывод: True
```

При преобразовании строк в логический тип любая непустая строка становится True



ValueError

Это встроенное исключение в Python, которое возникает, когда функция получает аргумент правильного типа, но с некорректным значением.

ValueError



Ответ

Например, если вы попытаетесь преобразовать строку, которая не может быть интерпретирована как число, в целое или вещественное число, будет вызвано исключение `ValueError`.

Пример

```
a = "abc"
b = float(a) # Ошибка: ValueError:
invalid literal for float() with base
10: 'abc'
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

```
result = 5
result += 3.14
print(type(result))
```

- a. <class 'int'>
- b. <class 'float'>
- c. Ошибка
- d. <class 'str'>



Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

```
result = 5
result += 3.14
print(type(result))
```

- a. <class 'int'>
- b. <class 'float'>**
- c. Ошибка
- d. <class 'str'>

Соотнесите функцию для преобразования типа с её результатом:

- | | |
|-------------------------------|-----------------------|
| 1. <code>int("10.0")</code> | a. <code>"10"</code> |
| 2. <code>float("3.14")</code> | b. <code>3.14</code> |
| 3. <code>str(10)</code> | c. <code>False</code> |
| 4. <code>bool(0)</code> | d. Ошибка |

Соотнесите функцию для преобразования типа с её результатом:

- | | |
|-------------------------------|-----------|
| 1. <code>int("10.0")</code> | d. Ошибка |
| 2. <code>float("3.14")</code> | b. 3.14 |
| 3. <code>str(10)</code> | a. "10" |
| 4. <code>bool(0)</code> | c. False |



ВОПРОСЫ



Домашнее задание

1. Калькулятор поездки

Напишите программу, которая получает от пользователя расстояние (в км), расход топлива (на 100 км) и цену топлива за литр (в евро), а выводит сколько литров топлива понадобится, и сколько будет стоить поездка.

Пример ввода:

Введите расстояние (км): 500

Введите расход топлива (л на 100 км): 8

Введите цену топлива за литр: 1.8

Пример вывода:

Для поездки потребуется топлива: 40.0

Стоимость поездки: 72.0

Домашнее задание

2. Размен суммы купюрами

Напишите программу, которая получает два числа от пользователя, умножает одно число на второе и выводит результат и оба числа через дефис. Не сохраняете результат умножения в переменной.

Пример ввода:

Введите сумму: 135

Введите номинал купюры: 50

Пример вывода:

Купюр нужно: 2

Остаток: 35

Заключение

